REQUIREMENTS ANALYSIS DOCUMENT
for
Automated Gherkin and BDD Test Scenario Generator

1.4

Oct 2026

Prepared by

Umut Akman

Yusuf Alperen Çelik

Melik Ahmet Caymazoğlu

Emircan Badur

TABLE OF CONTENTS

1 Introduction....4
1.1 Purpose of the system....4
1.2 Scope of the system....4
1.3 Objectives and success criteria of the project....4
1.3.1 Objectives....4
1.3.2 Success criteria....5
1.4 Definitions, acronyms, and abbreviations....5
1.5 References....6
1.6 Overview....6
2 Current system....6
2.1 Current process....6
3 Proposed system....7
3.1 Overview....7
3.2 Functional requirements....7
3.2.1 User Account Management....7
3.2.2 Backlog Import....8
3.2.3 Requirement Parsing....8
3.2.4 Requirement Validation....8
3.2.5 Scenario Generation....9
3.2.6 Output and Review....9
3.2.7 History....10
3.2.8 AI-Assisted Validation....10
3.3 Nonfunctional requirements....10
3.3.1 Usability....10
3.3.2 Reliability....10
3.3.3 Performance....11
3.3.4 Supportability....11
3.3.5 Implementation....11
3.3.6 Interface....11
3.3.7 Packaging....12
3.3.8 Legal....12
3.4 System models....12
3.4.1 Scenarios....12
3.4.2 Use case model....17
3.4.3 Object model....18
3.4.4 Dynamic model....19
3.4.4.1 Sequence diagram: Import, parse and validate backlog....19

3.4.4.2 Sequence diagram: Generate and display scenarios 21
3.4.4.3 Sequence diagram: Log in 23
3.4.5 User interfaces and Mock-ups 23
4 Glossary of Terms 23
5 Priority 25
6 Traceability 26

1 Introduction

The Automated Gherkin and BDD Test Scenario Generator is a developer-oriented tool that converts structured software requirements into Behavior-Driven Development (BDD) test scenarios written in Gherkin, using the Given–When–Then format. In its first version, the input is a Jira backlog exported as a CSV file. The system reads the Issue Type, the Summary, and, where available, the Issue key of each item, identifies the user stories, parses them into a structured internal model, and generates Gherkin feature files that developers can review, copy, and export.

1.1 Purpose of the system

The purpose of the system is to automate the transformation of user stories into BDD test scenarios. Writing Gherkin scenarios by hand is time consuming and potentially inconsistent between team members. The system reads user stories from a Jira backlog exported as a CSV file, validates them, and generates structured Gherkin feature files in the Given–When–Then format. By doing so, it helps developers and testers obtain consistent test scenarios earlier in the development process and reveals incomplete requirements before implementation begins.

1.2 Scope of the system

The system includes the following capabilities:

- Importing a Jira CSV file, identifying user stories by their Issue Type, and reading their Summary and, where available, their Issue key.
- Validating the parsed user stories and reporting clear feedback for incomplete or invalid input.
- Building a structured internal representation of each requirement.
- Generating one or more Gherkin scenarios per user story, including alternative and exceptional flows where they can be identified.
- Presenting the generated scenarios as valid Gherkin text that can be viewed, copied, and exported.
- User registration and login, with persistent storage of each user's imports and generated scenarios.
- Optional AI-assisted validation of the generated Gherkin text.

The system does not execute the generated scenarios, does not generate source code, and does not modify the Jira backlog.

1.3 Objectives and success criteria of the project

1.3.1 Objectives

- O-1: Convert user stories from a Jira CSV export into syntactically valid Gherkin feature files.
- O-2: Generate multiple scenarios for a requirement where alternative or exceptional behavior can be identified.
- O-3: Report clear and actionable feedback when a requirement is incomplete or cannot be processed.
- O-4: Store users, imports, and generated scenarios so that each user can review their history.

- O-5: Apply object-oriented analysis and design principles throughout the system.

1.3.2 Success criteria

- SC-1: For a Jira CSV file that contains user stories, the system identifies all items of type Story and generates at least one scenario for each.

- SC-2: Every generated feature file is accepted by a standard Gherkin parser without errors.

- SC-3: Every generated scenario can be traced back to its source user story, and every exported scenario identifies that user story by its Issue key or, when no Issue key is available, by its Summary.

- SC-4: A user story without a "so that" clause produces a warning instead of a failure.

- SC-5: A registered user can log in, import a CSV file, and see that import in their history.

- SC-6: A new input format can be added by writing a new parser and adapter, without modifying the scenario-generation engine.

1.4 Definitions, acronyms, and abbreviations

Term	Definition
User Story	In this document, a Jira backlog item of type Story. Its Summary contains a requirement.
User Story Template	A requirement format of the form "As a [role], I want [capability], so that [benefit]."
EARS	Easy Approach to Requirements Syntax, a set of sentence templates for writing requirements, e.g. "When [trigger], the system shall [response]."
Epic	A Jira issue type that groups related user stories.
Issue key	The unique identifier of a Jira issue, for example SHOP-10.
Feature file	A plain-text file with the .feature extension that contains a Feature and its Gherkin scenarios.
Import	A single processing of a submitted Jira CSV file, together with its user stories and generated scenarios, as stored in a user's history.
Jira	An issue-tracking tool used to manage backlogs.
BDD	Behavior-Driven Development, a development approach where behavior is described in a shared, readable format.
Gherkin	A plain-text language used to describe behavior with the keywords Feature, Scenario, Given, When, and Then.
CSV	Comma-Separated Values, a plain-text tabular file format.
AST	Abstract Syntax Tree, the structured internal representation of a parsed requirement.

1.5 References

1. Automated Gherkin and BDD Test Scenario Generator project description (problem statement), CSE443, 2026.
2. CSE443 Project Deliverable #1: Requirements Analysis Document, 2026.
3. Cucumber, "Gherkin Reference," https://cucumber.io/docs/gherkin/reference/.
4. Alistair Mavin, “EARS”, https://alistairmavin.com/ears/

1.6 Overview

This document specifies the requirements of the Automated Gherkin and BDD Test Scenario Generator and is organized into six sections. Section 1 introduces the system, its scope, and its objectives. Section 2 describes how BDD test scenarios are currently derived from backlog requirements, which motivates the proposed system. Section 3 is the main part of the document: it lists the functional and nonfunctional requirements and presents the analysis models, namely scenarios, the use case model, the object model, the dynamic model, and user interface mock-ups. Section 4 defines the terms used throughout the document. Section 5 assigns a priority to each requirement, and Section 6 traces the requirements back to the project problem statement. Readers who need only the expected behavior of the system can focus on Sections 3.2 and 3.4.2, while readers interested in the conceptual structure can refer to Section 3.4.3.

2 Current system

The Automated Gherkin and BDD Test Scenario Generator does not replace an existing software system. At present, BDD test scenarios are derived from backlog requirements manually by developers and testers. This section describes that manual process and its problems.

2.1 Current process

1. The team maintains its requirements as user stories in a Jira backlog. Each user story contains a requirement, typically written in the user story template or in EARS syntax.
2. A developer or tester reads each user story and interprets it to identify the actor, the intended action, the expected outcome, and any preconditions or triggers.
3. The same person writes the Given, When, and Then steps of one or more Gherkin scenarios by hand in a text editor or an IDE, and groups them into a feature file.
4. Alternative and exceptional flows, such as invalid input or missing information, are added only if the author thinks of them.
5. The author checks the syntax and wording of the scenarios manually or by running them through a Gherkin-aware tool, and the feature files are then stored alongside the project source.
6. If a link between a scenario and its source user story is needed, it is maintained by hand, for example by copying the Jira issue key into a tag or comment.

Problems of the current process

- Time-consuming: Every user story must be read, interpreted, and rewritten as scenarios by hand, which becomes costly as the backlog grows.
- Inconsistent results: Different authors phrase steps and structure scenarios differently, even for similar requirements.
- Missed alternative flows: Exceptional and boundary cases depend on the author's attention and are often omitted.

- Late detection of incomplete requirements: Missing roles, actions, or outcomes in a user story are usually discovered only when someone tries to write its scenarios, or later during testing.
- Weak traceability: Links between user stories and scenarios are easily lost or become outdated when stories change.
- No history: There is no record of which scenarios were produced from which version of the backlog, so earlier results are difficult to review or reproduce.

3 Proposed system

3.1 Overview

The proposed system is used by registered users who want to obtain Gherkin test scenarios from the requirements in a Jira backlog. A user registers, logs in, and submits a Jira CSV file. The system identifies the user stories in the file by their Issue Type and reads the requirement in each Summary, which may be written in the user story template or in EARS syntax. Where the file contains an Issue key column, the system also reads the Issue key of each user story. Each requirement is parsed into a common structured form and validated. Requirements that pass validation are transformed into one or more Gherkin scenarios, including alternative and exceptional flows where they can be identified. Requirements that are incomplete or unsupported are reported to the user together with the reason and a suggested correction.

The user can view the generated scenarios, copy them, and export them as feature files. The user can also see which user story each scenario was generated from. Exported feature files identify the source user story of each scenario by its Issue key or, when no Issue key is available, by a comment containing its Summary. Every processed file is stored in the user's history, so previous results can be reviewed later. Optionally, the generated scenarios can be checked by an AI component; the system works fully without it.

The functional requirements in Section 3.2 are grouped into the following modules: User Account Management, Backlog Import, Requirement Parsing, Requirement Validation, Scenario Generation, Output and Review, History, and AI-Assisted Validation.

3.2 Functional requirements

The functional requirements are grouped by module. Each requirement has a unique identifier and is written in EARS syntax.

3.2.1 User Account Management

ID	Requirement
FR-1	When a visitor submits an email address and a password for registration, the system shall create a user account.
FR-2	If a registration request contains an email address that is already registered, then the system shall reject the request and display an error message.
FR-3	When a registered user submits valid credentials, the system shall authenticate the user and grant access to the system's functions.

ID	Requirement
FR-4	If a login request contains invalid credentials, then the system shall deny access and display an error message.
FR-5	While a user is not authenticated, the system shall restrict access to the import, generation, and history functions.
FR-6	When an authenticated user requests to log out, the system shall end the user's session.

3.2.2 Backlog Import

ID	Requirement
FR-7	When an authenticated user submits a Jira CSV file, the system shall read the Issue Type and the Summary of each item in the file.
FR-8	Where the submitted file contains an Issue key column, the system shall read the Issue key of each item.
FR-9	The system shall identify an item as a user story when its Issue Type is "Story".
FR-10	The system shall exclude items whose Issue Type is not "Story" from scenario generation.
FR-11	If a submitted file cannot be read as CSV or lacks an Issue Type column or a Summary column, then the system shall reject the file and display the reason.
FR-12	If a submitted file contains no items of type "Story", then the system shall inform the user that no user stories were found.

3.2.3 Requirement Parsing

ID	Requirement
FR-13	When a user story has been identified, the system shall determine whether its Summary is written in the user story template or in EARS syntax.
FR-14	When a Summary is written in the user story template, the system shall extract the role, the capability, and the benefit, if present.
FR-15	When a Summary is written in EARS syntax, the system shall extract the precondition, the trigger, and the system response, where present.
FR-16	If a Summary matches neither the user story template nor EARS syntax, then the system shall mark the user story as unsupported and report it to the user.
FR-17	The system shall represent every parsed requirement in a common structured form that is independent of its original format.

3.2.4 Requirement Validation

ID	Requirement
FR-18	When a requirement has been parsed, the system shall check that its required elements are present: the role and the capability for the user story template, and the system response for EARS syntax.

ID	Requirement
FR-19	If a requirement in the user story template has no benefit, then the system shall issue a warning and continue processing it.
FR-20	If a required element of a requirement is missing, then the system shall report an error that identifies the user story, the missing element, and a suggested correction.
FR-21	The system shall not generate scenarios for a requirement that has failed validation.
FR-22	The system shall distinguish, in the messages it displays, between errors caused by invalid input and errors caused by internal failures.

3.2.5 Scenario Generation

ID	Requirement
FR-23	When a requirement has passed validation, the system shall generate at least one Gherkin scenario for it.
FR-24	The system shall include in each generated scenario a feature name, a scenario title, at least one When step, at least one Then step, and Given steps where the requirement defines preconditions.
FR-25	When alternative or exceptional behavior can be identified from a requirement, the system shall generate an additional scenario for each identified behavior.
FR-26	The system shall associate each generated scenario with the user story from which it was generated.

3.2.6 Output and Review

ID	Requirement
FR-27	When scenario generation completes, the system shall display the generated scenarios as Gherkin text.
FR-28	When a user requests to copy generated Gherkin text, the system shall make the text available for copying.
FR-29	When a user requests to export generated scenarios, the system shall produce a feature file containing valid Gherkin syntax.
FR-30	Where the Issue key of a user story is available, the system shall tag each exported scenario generated from that user story with the Issue key.
FR-31	If the Issue key of a user story is not available, then the system shall precede each exported scenario generated from that user story with a comment containing the Summary of the user story.
FR-32	When a user selects a generated scenario, the system shall display the user story from which it was generated.

3.2.7 History

ID	Requirement
FR-33	When a submitted file has been processed, the system shall store the import, its user stories, and the generated scenarios in the history of the user who submitted it.
FR-34	When an authenticated user requests their history, the system shall display the user's previous imports.
FR-35	When a user selects a previous import, the system shall display the scenarios generated for it.
FR-36	The system shall not display a user's history to any other user.

3.2.8 AI-Assisted Validation

ID	Requirement
FR-37	Where AI-assisted validation is enabled, the system shall submit each generated scenario to the AI component for a content check and display the findings.
FR-38	If the AI component is unavailable or returns an error, then the system shall complete scenario generation without AI validation and notify the user that the check was skipped.

3.3 Nonfunctional requirements

The nonfunctional requirements describe the quality attributes and constraints of the system that are not directly related to its functionality. They are grouped under the categories of usability, reliability, performance, supportability, implementation, interface, packaging, and legal requirements. Each requirement has a unique identifier of the form NFR-n and is written in EARS syntax.

3.3.1 Usability

ID	Requirement
NFR-1	The system shall allow a logged-in user to submit a Jira CSV file and view the generated scenarios in no more than three user actions.
NFR-2	The system shall present all interface text and messages in English.

3.3.2 Reliability

ID	Requirement
NFR-3	If one user story cannot be processed, then the system shall continue processing the remaining user stories in the same file.
NFR-4	If an unexpected internal error occurs, then the system shall display an error message and remain usable instead of terminating abnormally.
NFR-5	The system shall keep a user's stored history available after the application is restarted.

ID	Requirement
NFR-6	The system shall store user passwords only in hashed form.

3.3.3 Performance

ID	Requirement
NFR-7	When a user submits a Jira CSV file containing up to 100 items, the system shall complete parsing, validation, and scenario generation within 10 seconds, excluding AI-assisted validation.

3.3.4 Supportability

ID	Requirement
NFR-8	The system shall allow support for a new requirement format to be added by implementing a new parser and adapter, without modifying the scenario-generation components.
NFR-9	The system shall allow a new validation rule to be added without modifying the existing validation rules.
NFR-10	The system shall keep requirement parsing, validation, scenario generation, and output formatting in separate components.

3.3.5 Implementation

ID	Requirement
NFR-11	The system shall be implemented in Java.
NFR-12	The system shall store its data in an embedded relational database that does not require a separate database server.
NFR-13	The system shall use the Chain of Responsibility pattern for its validation and processing steps, the Adapter pattern to convert each input format into the common requirement form, and the Builder pattern to construct Gherkin scenarios.

3.3.6 Interface

ID	Requirement
NFR-14	The system shall accept Jira CSV files in which fields are separated by commas, text fields may be enclosed in double quotes, and the text is encoded in UTF-8.
NFR-15	The system shall produce feature files in Gherkin syntax with the .feature extension.
NFR-16	The system shall communicate with the AI component only through a single interface that allows the AI provider to be replaced.

3.3.7 Packaging

ID	Requirement
NFR-17	The system shall be delivered as a GitHub repository containing the source code, the documentation, and instructions for building and running the system.
NFR-18	The system shall have AI-assisted validation disabled in its default configuration.

3.3.8 Legal

ID	Requirement
NFR-19	The system shall store only the personal data needed for its functions: the user's email address and password hash.
NFR-20	While AI-assisted validation is disabled, the system shall not send any data to the AI component.

3.4 System models

This section presents the analysis models of the system. Section 3.4.1 describes concrete scenarios of how users work with the system. Section 3.4.2 presents the use case model, which shows the actors and the functions they use. Section 3.4.3 presents the object model, which describes the concepts of the problem domain and their relationships. Section 3.4.4 presents the dynamic model, which shows how the objects of the system collaborate over time. Section 3.4.5 presents the mock-ups of the user interface. The models use the same names for actors, use cases, and classes, and they realize the functional requirements of Section 3.2.

3.4.1 Scenarios

This section describes concrete scenarios of how users work with the system. The scenarios use a small example backlog of an online shop, which is described in scenario S-2, and a fictional QA engineer named Elif. They illustrate the functional requirements of Section 3.2 and are the basis of the use cases in Section 3.4.2.

Scenario	**S-1: Registration and login**
Participating actors	Elif, a QA engineer (Visitor, then Registered user)
Entry condition	Elif has no user account yet.
Flow of events	1. Elif opens the system. Because Elif is not logged in, the system restricts the import, generation, and history functions. 2. Elif submits the email address elif@example.com and a password for registration. 3. The system creates a user account for Elif. 4. Elif submits the same email address and password to log in.

Scenario	**S-1: Registration and login**
	5. The system authenticates Elif and grants access to the import, generation, and history functions. 6. When Elif has finished working, Elif requests to log out and the system ends the session.
Alternative flows	A1. Elif tries to register with an email address that is already registered. The system rejects the request and displays an error message. A2. Elif submits a wrong password at login. The system denies access and displays an error message.
Related requirements	FR-1 – FR-6

Scenario	**S-2: Generating Gherkin scenarios from a Jira backlog**
Participating actors	Elif (Registered user)
Entry condition	Elif is logged in and has the Jira CSV file shop-backlog.csv, exported from the Jira project SHOP of an online shop. The file contains five items: SHOP-1, Epic: "Shopping Cart" SHOP-2, Story: "As a customer, I want to add a product to my cart, so that I can buy it later." SHOP-3, Story: "As a customer, I want to remove a product from my cart, so that I can correct a mistake." SHOP-4, Story: "As a customer, I want to see the total price of my cart." SHOP-5, Story: "As a visitor, I want to search for a product, so that I can find it quickly."
Flow of events	1. Elif selects the file and submits it. 2. The system reads the Issue Type and the Summary of each of the five items. Because the file contains an Issue key column, the system also reads the Issue key. 3. The system identifies the four items of type Story as user stories and excludes the Epic SHOP-1 from scenario generation. 4. For each user story, the system determines that the Summary is written in the user story template and extracts the role, the capability, and the benefit. 5. The system validates each requirement. SHOP-4 has no "so that" clause and therefore no benefit, so the system issues a warning for it and continues processing it. 6. The system generates at least one Gherkin scenario for each requirement. For SHOP-2 it generates the scenario "Customer adds a product to the cart" with the steps "Given the customer is viewing a

Scenario	S-2: Generating Gherkin scenarios from a Jira backlog
	product", "When the customer adds the product to the cart", and "Then the product is in the customer's cart". It adds further scenarios for the alternative flows that it can identify. 7. The system stores the import in Elif's history and displays the generated scenarios as Gherkin text, together with the warning for SHOP-4.
Alternative flows	Incomplete or unsupported requirements are described in scenario S-3. Files that cannot be processed are described in scenario S-4.
Related requirements	FR-7 – FR-10, FR-13, FR-14, FR-17 – FR-19, FR-23 – FR-27, FR-33

Scenario	S-3: Handling EARS, incomplete, and unsupported requirements
Participating actors	Elif (Registered user)
Entry condition	Elif is logged in and has a small Jira CSV file with three items of type Story.
Flow of events	1. Elif submits the file. Its three user stories are SHOP-6, "When a user enters a wrong password three times, the system shall lock the account.", SHOP-7, "As a customer, so that I know my order was received.", and SHOP-8, "Improve the performance." 2. The system identifies the three user stories and determines the format of each Summary. 3. SHOP-6 is written in EARS syntax, a template of the form "When [trigger], the system shall [response]." The system extracts the trigger ("a user enters a wrong password three times") and the system response ("lock the account"). The requirement passes validation and the system generates a scenario for it. Because the trigger contains a limit, the system also generates an additional scenario in which the account is not locked after fewer than three wrong attempts. 4. SHOP-7 is written in the user story template but has no capability. The system reports an error that identifies the user story, the missing element (the capability), and a suggested correction (add an "I want ..." clause). The system generates no scenarios for SHOP-7. 5. SHOP-8 matches neither the user story template nor EARS syntax. The system marks it as unsupported and reports it to Elif. 6. The system continues processing despite these problems. It displays the scenarios for SHOP-6 and the messages for SHOP-7 and SHOP-8, which state that the cause is invalid input and not an internal failure.
Related requirements	FR-13 – FR-16, FR-18, FR-20 – FR-22, FR-25, NFR-3

Scenario	**S-4: Rejecting files that cannot be processed**
Participating actors	Elif (Registered user)
Entry condition	Elif is logged in and has two files: notes.csv, which has no Summary column, and epics-only.csv, which contains only Epics.
Flow of events	1. Elif submits notes.csv. 2. The system rejects the file and displays the reason: the required Summary column is missing. 3. Elif submits epics-only.csv. 4. The system reads the file and excludes all of its items, because none of them has the Issue Type Story. The system informs Elif that no user stories were found.
Related requirements	FR-7, FR-9 – FR-12

Scenario	**S-5: Copying and exporting scenarios**
Participating actors	Elif (Registered user)
Entry condition	Elif has generated the scenarios from shop-backlog.csv (scenario S-2) and the result is displayed.
Flow of events	1. Elif selects the scenario "Customer adds a product to the cart" and requests to copy it. The system makes the Gherkin text available for copying. 2. Elif requests to export all generated scenarios. 3. The system produces a feature file with valid Gherkin syntax. Because the Issue keys are available, the system tags each exported scenario with the Issue key of its user story, for example @SHOP-2.
Alternative flows	A1. Elif exports scenarios that were generated from a file without an Issue key column. The system precedes each exported scenario with a comment that contains the Summary of its user story, for example "# Source: As a customer, I want to add a product to my cart, so that I can buy it later."

Example of the exported feature file	`Feature: Shopping cart`
	
	`@SHOP-2`
	`Scenario: Customer adds a product to the cart`
	`Given the customer is viewing a product`
	`When the customer adds the product to the cart`
	`Then the product is in the customer's cart`

Scenario	**S-5: Copying and exporting scenarios**

	`@SHOP-3`
	`Scenario: Customer removes a product from the cart`
	`Given the customer's cart contains a product`
	`When the customer removes the product from the cart`
	`Then the product is no longer in the customer's cart`

Related requirements	FR-8, FR-28 – FR-31

Scenario	**S-6: Reviewing the history**
Participating actors	Elif and Can (Registered users)
Entry condition	Elif imported shop-backlog.csv (scenario S-2) on the previous day. Can is another registered user with their own imports.
Flow of events	1. Elif logs in and requests the history. 2. The system displays Elif's previous imports. 3. Elif selects the import of shop-backlog.csv. The system displays the scenarios that were generated for it. 4. Elif selects the scenario "Customer adds a product to the cart". The system displays its source user story, SHOP-2, with its Summary. 5. Later, Can logs in and requests the history. The system displays only Can's own imports and none of Elif's imports.
Related requirements	FR-32, FR-34 – FR-36

Scenario	**S-7: AI-assisted validation (optional)**
Participating actors	Elif (Registered user), AI component
Entry condition	AI-assisted validation is enabled in the configuration. It is disabled by default. Elif has submitted shop-backlog.csv (scenario S-2).
Flow of events	1. After the scenarios are generated, the system submits each generated scenario to the AI component for a content check. 2. The AI component returns its findings, for example that the step "Then the system works" is too vague to be verified. 3. The system displays the findings together with the scenarios.
Alternative flows	A1. The AI component is unavailable or returns an error. The system completes scenario generation without AI validation and notifies Elif that the check was skipped.

Scenario	**S-7: AI-assisted validation (optional)**
	A2. AI-assisted validation is disabled, which is the default. The system does not send any data to the AI component and displays no findings.
Related requirements	FR-37, FR-38, NFR-18, NFR-20

3.4.2 Use case model

Figure 3.4.2-1 shows the use case model of the system. The system has three actors. A Visitor is a person who is not logged in and can register an account (Register Account) and log in (Log In). A Registered user is a person who has logged in and can log out, generate scenarios from a backlog, copy or export the generated scenarios, view the source user story of a scenario, and view their history. The AI component is a secondary actor that takes part only in the optional use case Validate Scenarios with AI.

The use case Generate Scenarios from Backlog includes Parse Requirements and Validate Requirements, because the generation of scenarios always depends on both. Validate Scenarios with AI extends Generate Scenarios from Backlog and is performed only when AI-assisted validation is enabled. All use cases except Register Account and Log In require that the user is logged in.

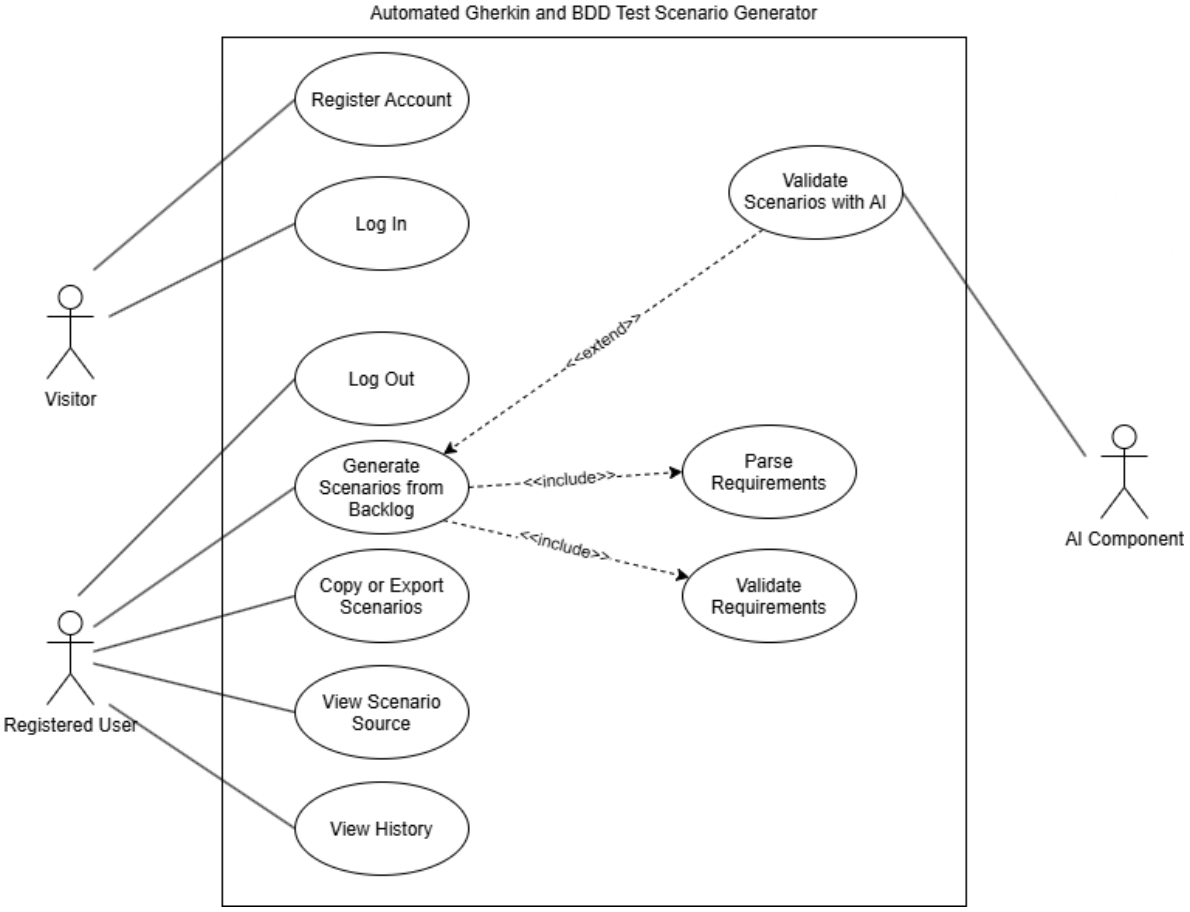


Figure 3.4.2-1

3.4.3 Object model

A RegisteredUser submits any number of Imports. An Import contains the UserStories read from one Jira CSV file. A UserStory has an optional issue key, its summary, and a status. It is parsed into at most one Requirement, which is either a UserStoryTemplateRequirement (role, capability, and an optional benefit) or an EARSRequirement (an optional precondition, an optional trigger, and a system response). A UserStory that is unsupported has no Requirement. The validation of a user story produces any number of ValidationResults, each of which is an error or a warning.

A user story that has passed validation is described by one Feature, which contains one or more Scenarios. A Scenario consists of an ordered list of Steps, is either a normal or an alternative flow, and may be checked by AI findings. A scenario is associated with its source user story through its Feature (FR-26, FR-32).

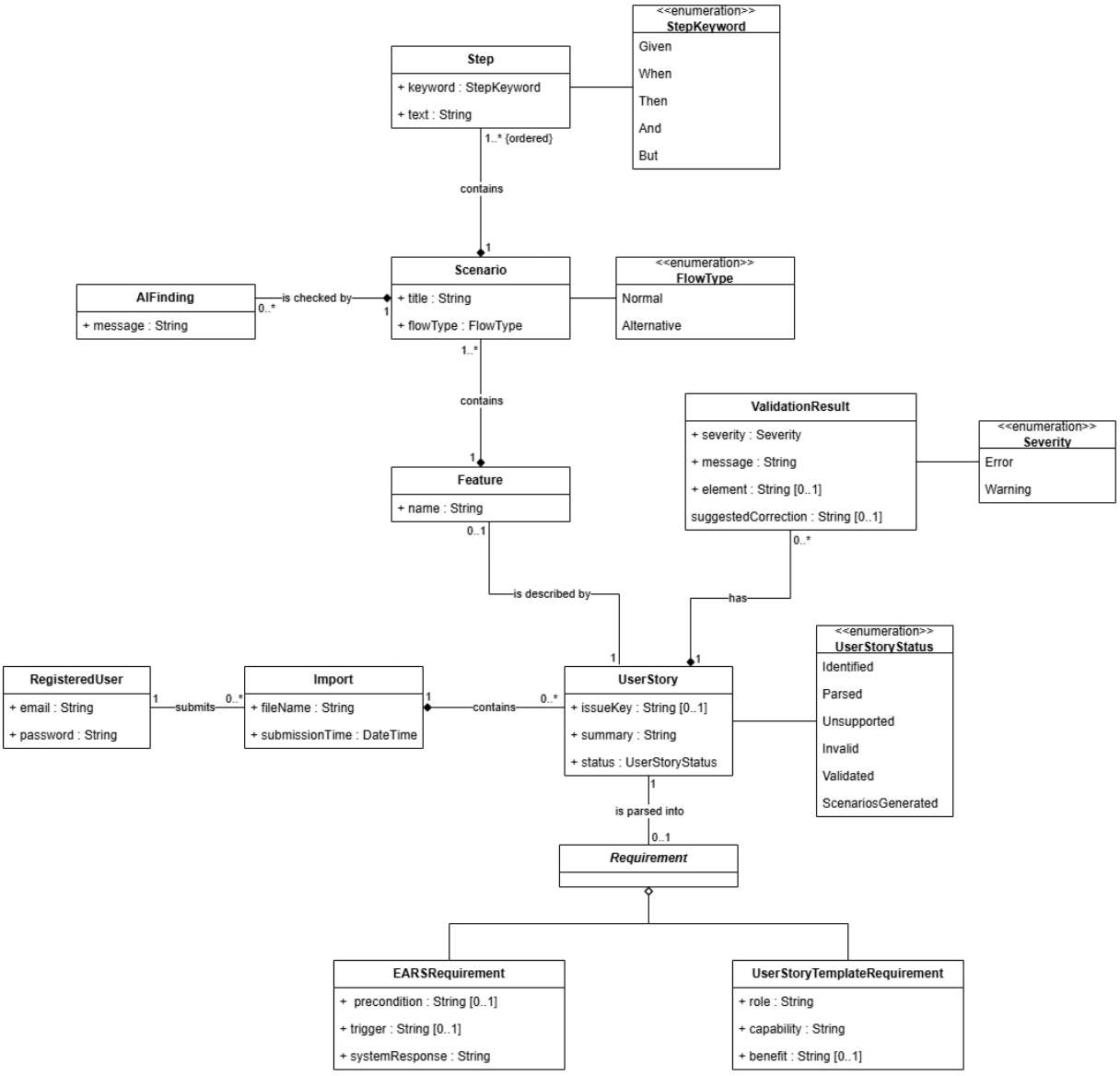


Figure 3.4.3-1

3.4.4 Dynamic model

The dynamic model shows how the objects of the system collaborate over time to realize the main use cases. It consists of three sequence diagrams. Following the analysis convention, boundary objects (ImportForm, LoginForm) handle the interaction with the user, control objects (GenerationControl, AccountControl) coordinate a use case, and entity objects (Import, UserStory, Requirement, Feature, Scenario, RegisteredUser) are the classes of the object model. Figures 3.4.4-1 and 3.4.4-2 describe the use case Generate Scenarios from Backlog together with Parse Requirements, Validate Requirements, and Validate Scenarios with AI. Figure 3.4.4-3 describes the use case Log In.

3.4.4.1 Sequence diagram: Import, parse and validate backlog

Figure 3.4.4-1 shows how a submitted Jira CSV file is processed up to the validation of its requirements. The user submits the file through the ImportForm, which passes it to the GenerationControl. The GenerationControl reads the items of the file. If the file cannot be read or a required column is missing, it reports the reason to the user and the processing ends (FR-11).

Otherwise, the GenerationControl creates an Import and, for each item of Issue Type "Story", a UserStory (FR-9, FR-10). It then determines the format of the Summary and creates a UserStoryTemplateRequirement or an EARSRequirement. If the Summary matches neither format, the user story is marked as unsupported and an error is added (FR-16). For every created Requirement, the GenerationControl checks the required elements. If a required element is missing, an error is added and the user story is marked as invalid (FR-20); if only the benefit is missing, a warning is added and processing continues (FR-19). The processing then continues with the next user story (NFR-3). Figure 3.4.4-2 continues this sequence.

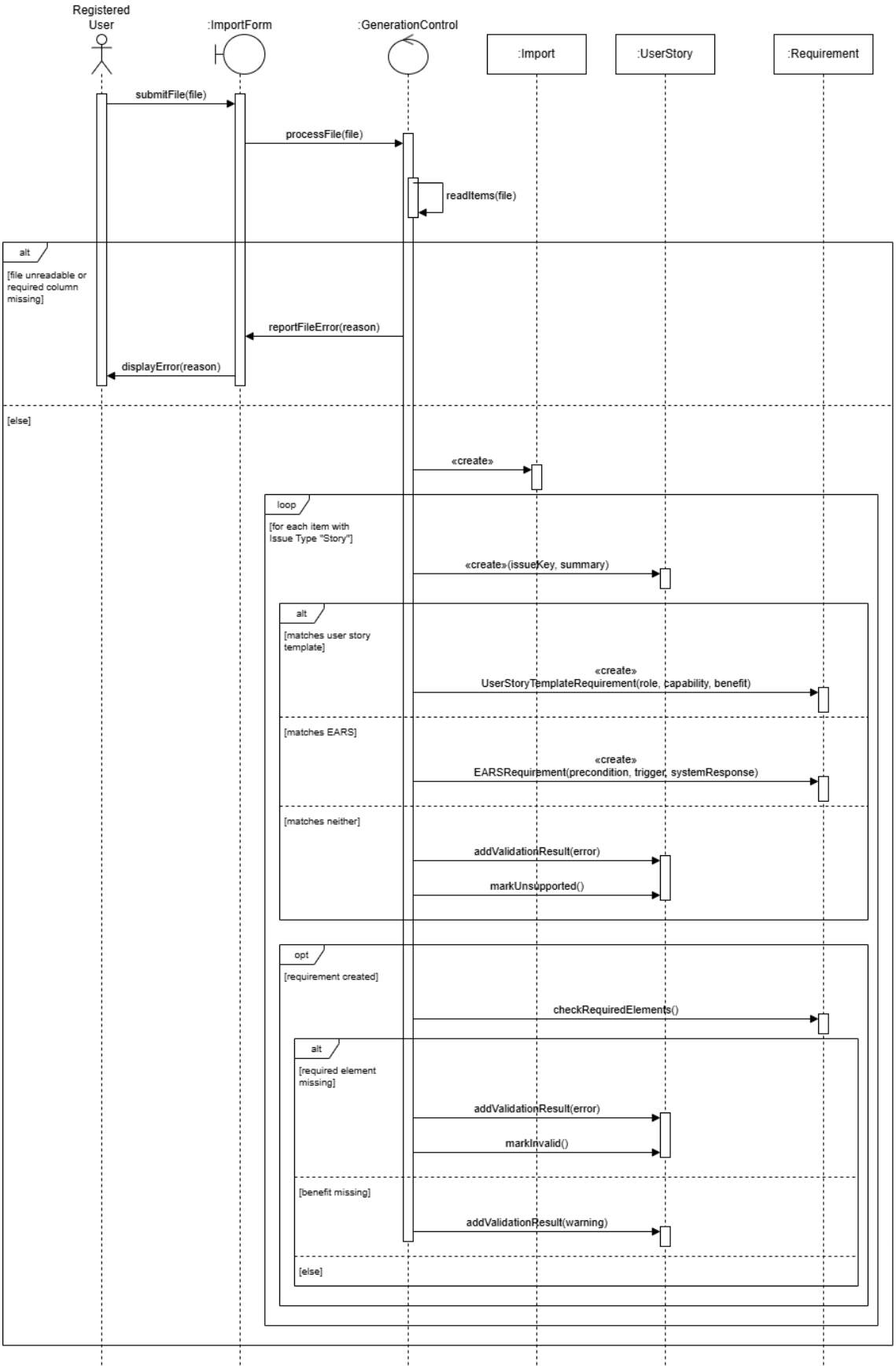

Registered User
:ImportForm
:GenerationControl
:Import
:UserStory
:Requirement
submitFile(file)
processFile(file)
readItems(file)
alt
[file unreadable or required column missing]
reportFileError(reason)
displayError(reason)
[else]
«create»
loop
[for each item with Issue Type "Story"]
«create»(issueKey, summary)
alt
[matches user story template]
«create»
UserStoryTemplateRequirement(role, capability, benefit)
[matches EARS]
«create»
EARSRequirement(precondition, trigger, systemResponse)
[matches neither]
addValidationResult(error)
markUnsupported()
opt
[requirement created]
checkRequiredElements()
alt
[required element missing]
addValidationResult(error)
markInvalid()
[benefit missing]
addValidationResult(warning)
[else]

Figure 3.4.4-1

3.4.4.2 Sequence diagram: Generate and display scenarios

Figure 3.4.4-2 continues Figure 3.4.4-1. For each user story that passed validation, the GenerationControl creates a Feature with the feature name and a Scenario for the normal flow (FR-23, FR-24). Where alternative or exceptional behavior can be identified, it creates an additional Scenario for each identified behavior (FR-25). The scenarios are added to the Feature, and the Feature is associated with the user story (FR-26). The user stories, together with their features and scenarios, are then stored in the Import and become part of the user's history (FR-33).

If AI-assisted validation is enabled, the GenerationControl submits each Scenario to the AI component and adds the findings it returns to the Scenario (FR-37). If the AI component is unavailable or returns an error, the AI validation is skipped and the processing continues (FR-38). Finally, the ImportForm displays the generated scenarios as Gherkin text together with the warnings, the errors, the AI findings, or a notice that the AI check was skipped (FR-27).

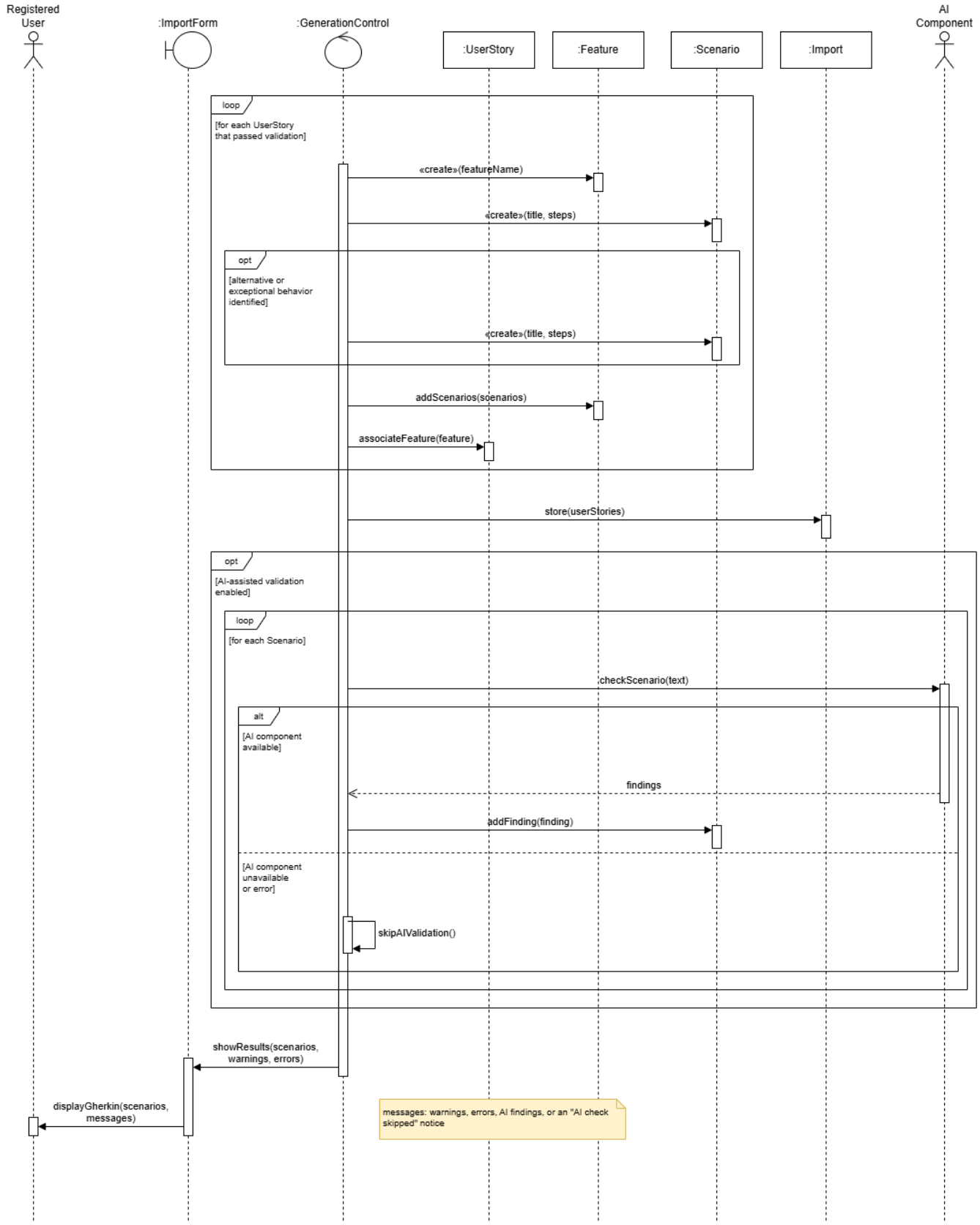

Registered User
:ImportForm
:GenerationControl
:UserStory
:Feature
:Scenario
:Import
AI Component
loop
[for each UserStory that passed validation]
«create»(featureName)
«create»(title, steps)
opt
[alternative or exceptional behavior identified]
«create»(title, steps)
addScenarios(soenarios)
associateFeature(feature)
store(userStories)
opt
[AI-assisted validation enabled]
loop
[for each Scenario]
checkScenario(text)
alt
[AI component available]
findings
addFinding(finding)
[AI component unavailable or error]
skipAIValidation()
showResults(scenarios, warnings, errors)
displayGherkin(scenarios, messages)
messages: warnings, errors, AI findings, or an "AI check skipped" notice

Figure 3.4.4-2

3.4.4.3 Sequence diagram: Log in

Figure 3.4.4-3 shows the use case Log In. The visitor enters an email address and a password in the LoginForm, which passes them to the AccountControl. The AccountControl verifies the credentials against the RegisteredUser. If the credentials are valid, the AccountControl grants access and the LoginForm displays the main page (FR-3). Otherwise, the AccountControl denies access and the LoginForm displays an error message (FR-4).

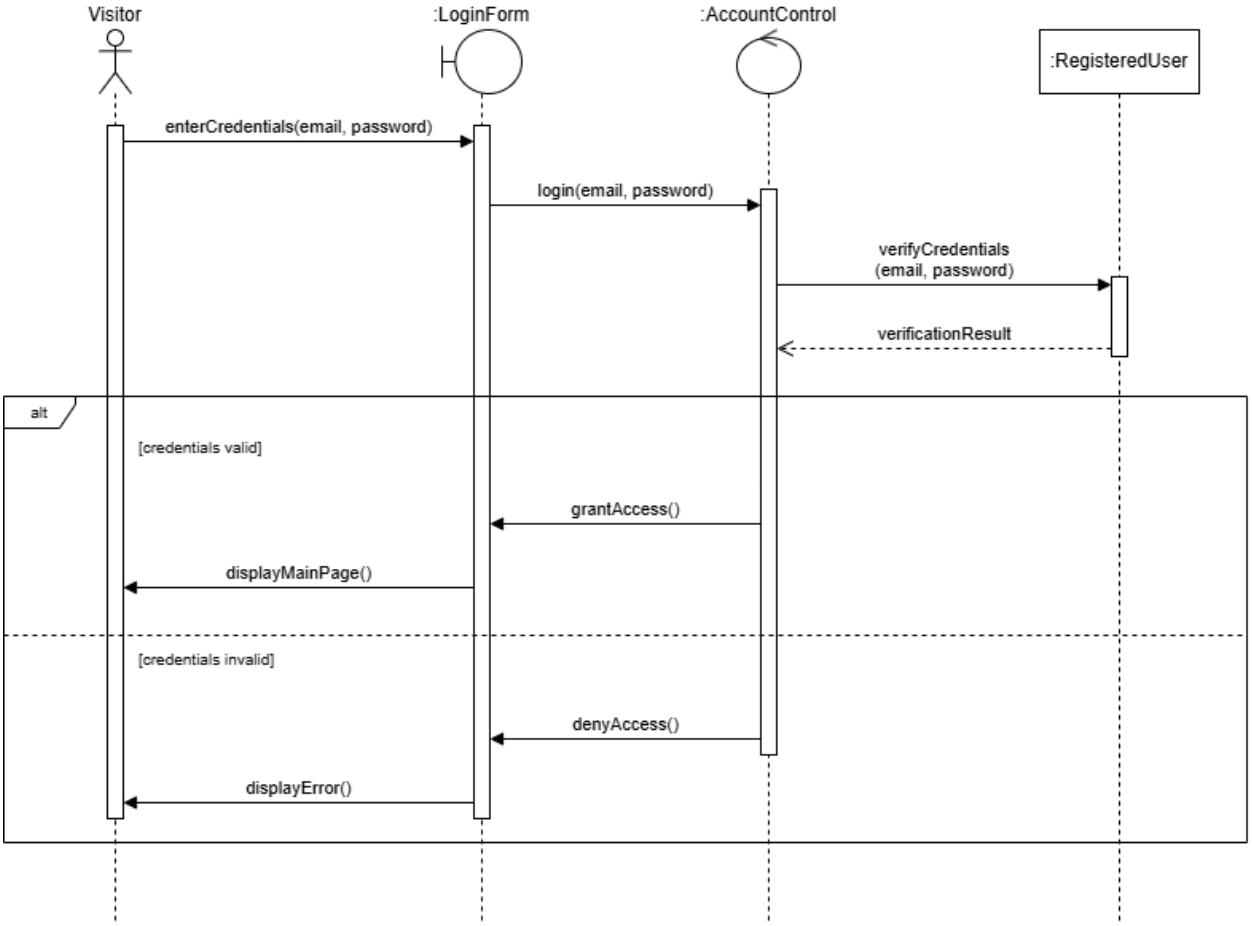


Figure 3.4.4-3

3.4.5 User interfaces and Mock-ups

4 Glossary of Terms

This glossary lists the important terms used in this document in alphabetical order, so that the requirements and models use the same vocabulary. Terms that are also defined in Section 1.4 have the same definition here.

Term	Definition
Adapter	A component that converts the result of a format-specific parser into the common requirement form.
AI component	An optional component that checks the content of generated scenarios and reports its findings.
Alternative flow	Behavior of the system that differs from the normal successful behavior described by a requirement, such as the handling of invalid input or missing information.
AST	Abstract Syntax Tree, the structured internal representation of a parsed requirement.
BDD	Behavior-Driven Development, a development approach where behavior is described in a shared, readable format.
Benefit	The value or reason a role expects from a capability, introduced by "so that" in the user story template.
Capability	What a role wants the system to provide, introduced by "I want" in the user story template.
Common requirement form	The structured representation of a parsed requirement that does not depend on the format in which the requirement was originally written.
CSV	Comma-Separated Values, a plain-text tabular file format.
EARS	Easy Approach to Requirements Syntax, a set of sentence templates for writing requirements, e.g. "When [trigger], the system shall [response]."
Epic	A Jira issue type that groups related user stories.
Error	A validation result stating that a requirement cannot be processed, so that no scenarios are generated for it.
Feature	The Gherkin element that names a capability of the system and groups the scenarios that describe it.
Feature file	A plain-text file with the .feature extension that contains a Feature and its Gherkin scenarios.
Gherkin	A plain-text language used to describe behavior with the keywords Feature, Scenario, Given, When, and Then.
Issue key	The unique identifier of a Jira issue, for example SHOP-10.
Issue Type	The Jira field that classifies an item, for example Story or Epic.
Jira	An issue-tracking tool used to manage backlogs.
Parser	A component that reads a requirement written in one format and extracts its elements.
Requirement	A statement of a capability or behavior that the system shall provide. In this document, the text in the Summary of a user story.
Role	The kind of user for whom a user story is written, introduced by "As a" in the user story template.

Term	Definition
Scenario	A concrete example of the system's behavior, written in Gherkin as a sequence of Given, When, and Then steps.
Step	A line of a Gherkin scenario that starts with Given (a precondition), When (an action or trigger), or Then (an expected outcome).
Summary	The Jira field that contains the short text of an item. For a user story, it holds the requirement.
System response	The behavior that the system shall perform when the conditions of a requirement are met, expressed in EARS as "the system shall ...".
Trigger	An event that causes the system to respond, introduced by "When" in EARS syntax.
Unsupported requirement	A requirement whose Summary matches neither the user story template nor EARS syntax.
User Story	In this document, a Jira backlog item of type Story. Its Summary contains a requirement.
User Story Template	A requirement format of the form "As a [role], I want [capability], so that [benefit]."
Validation	The check that a parsed requirement contains all of its required elements.
Warning	A validation result that reports a problem in a requirement but allows its processing to continue.

5 Priority

The requirements are prioritized using the MoSCoW scheme with the levels Must, Should, and Could. The level Won't is not used, because what the system does not do is stated in Section 1.2. The priorities are assigned by the project team and may be revised as the project progresses.

Priority	Meaning	Functional requirements	Nonfunctional requirements
Must	Required to meet the success criteria or demanded by the problem statement. The system is not acceptable without it.	FR-1, FR-3 – FR-5, FR-7, FR-8, FR-9 – FR-11, FR-13 – FR-21, FR-23, FR-24, FR-25,FR-26, FR-27, FR-28, FR-29 – FR-31, FR-32, FR-33, FR-34, FR-36	NFR-5, NFR-6, NFR-8, NFR-10 – NFR-15, NFR-17
Should	Important for a complete system, but the core transformation still works without it.	FR-2, FR-6, FR-12, FR-22, FR-35	NFR-1 – NFR-4, NFR-7, NFR-9, NFR-19

Priority	Meaning	Functional requirements	Nonfunctional requirements
Could	Desirable and optional. Implemented only if time allows.	FR-37, FR-38	NFR-16, NFR-18, NFR-20

6 Traceability

Each requirement in this document traces to a section of the project problem statement [1]. The sections of the problem statement are numbered in their order of appearance, for example 3.1 is Requirement Input and 3.11 is Extensibility. Requirements that come from decisions of the project team rather than from a specific section of the problem statement are listed in the last row of the table.

Problem statement section	Requirements
2 Objective Overview	NFR-8, NFR-10, NFR-13
3.1 Requirement Input	FR-7, FR-11, FR-12, FR-13, FR-16, NFR-14
3.2 Requirement Parsing	FR-13 – FR-16, NFR-8
3.3 Requirement Abstract Syntax Tree	FR-17
3.4 Requirement Validation	FR-18 – FR-20, NFR-9, NFR-13
3.5 Input Adaptation	FR-17, NFR-8, NFR-13
3.6 BDD Scenario Generation	FR-23, FR-24
3.7 Gherkin Scenario Construction	FR-24, NFR-13
3.8 Scenario Formatting and Output	FR-26 – FR-29, FR-32, NFR-10, NFR-15
3.9 Multiple Scenarios and Alternative Flows	FR-25
3.10 Error Handling and User Feedback	FR-11, FR-16, FR-20 – FR-22, NFR-3, NFR-4
3.11 Extensibility	FR-37, FR-38, NFR-8, NFR-9, NFR-16
4 Technical Architecture and Database Requirements	FR-26, FR-33 – FR-35, NFR-5, NFR-10, NFR-12, NFR-13
5 Challenges and Constraints	NFR-10, NFR-13
Project team decisions (no direct source in the problem statement)	FR-1 – FR-6, FR-8 – FR-10, FR-30, FR-31, FR-36, NFR-1, NFR-2, NFR-6, NFR-7, NFR-11, NFR-17 – NFR-20